UNIFECAF · ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Sistema Automotivo — Gestão de Estoque de Veículos

Parte Prática — Sistema funcionando e códigos principais

Aluno	Breno Silva de Abreu
Matrícula	189836
Disciplina	Object Oriented Programming
Tecnologias	Java 21 · Spring Boot 3.3 · MySQL 8
Repositório	https://github.com/PharmaBR/sistema-automotivo-unifecaf
Data	Outubro de 2026

1. Arquitetura

Código-fonte: `https://github.com/PharmaBR/sistema-automotivo-unifecaf`

O sistema segue o padrão **MVC** em camadas, exposto como **API REST**:

- **Model** — `domain/`: entidades JPA com as regras de negócio.
- **Controller** — `web/`: recebem HTTP, validam a entrada (DTOs) e delegam aos serviços.
- **View** — a API responde JSON; o front-end em `static/` consome essa API no navegador.
- **Service** e **Repository** — orquestram os casos de uso e acessam o MySQL via Spring Data JPA.

Fluxo de uma requisição: `Controller → Service → Entidade (regras) → Repository → MySQL`.

Estrutura de pastas

```
sistema-automotivo/
├── pom.xml                     dependências (Maven)
├── Dockerfile / docker-compose.yml
├── docs/api.http               requisições de demonstração
└── src/
    ├── main/java/br/edu/unifecaf/automotivo/
    │   ├── AutomotivoApplication.java
    │   ├── domain/       BaseEntity, Brand, VehicleModel, Vehicle,
    │   │                 VehicleStatus, VehicleCategory, exceções
    │   ├── repository/   BrandRepository, VehicleModelRepository,
    │   │                 VehicleRepository, VehicleFilter, VehicleSpecifications
    │   ├── service/      BrandService, VehicleModelService, VehicleService
    │   ├── web/          BrandController, VehicleModelController,
    │   │                 VehicleController, GlobalExceptionHandler, dto/
    │   └── config/       OpenApiConfig, DataSeeder
    ├── main/resources/
    │   ├── application.properties
    │   ├── db/schema.sql modelo físico MySQL
    │   └── static/       front-end (index.html, css, js)
    └── test/java/...     VehicleTest (unidade), VehicleApiIntegrationTest
```

Endpoints

Método	Rota	Função
GET	/api/vehicles	Consulta com filtros: brandId, modelId, brand, model, category, status, minPrice, maxPrice, minYear, maxYear, color; paginação e ordenação
GET	/api/vehicles/{id}	Detalhe de um veículo
GET	/api/vehicles/summary	Quantidade por status
POST	/api/vehicles	Cadastro

Método	Rota	Função
PATCH	/api/vehicles/{id}	Atualiza preço, quilometragem, status e/ou cor
DELETE	/api/vehicles/{id}	Remove (só vendidos/descontinuados)
GET/POST/PUT/DELETE	/api/brands	CRUD de marcas
GET/POST/PUT/DELETE	/api/models	CRUD de modelos (filtro ?brandId=)

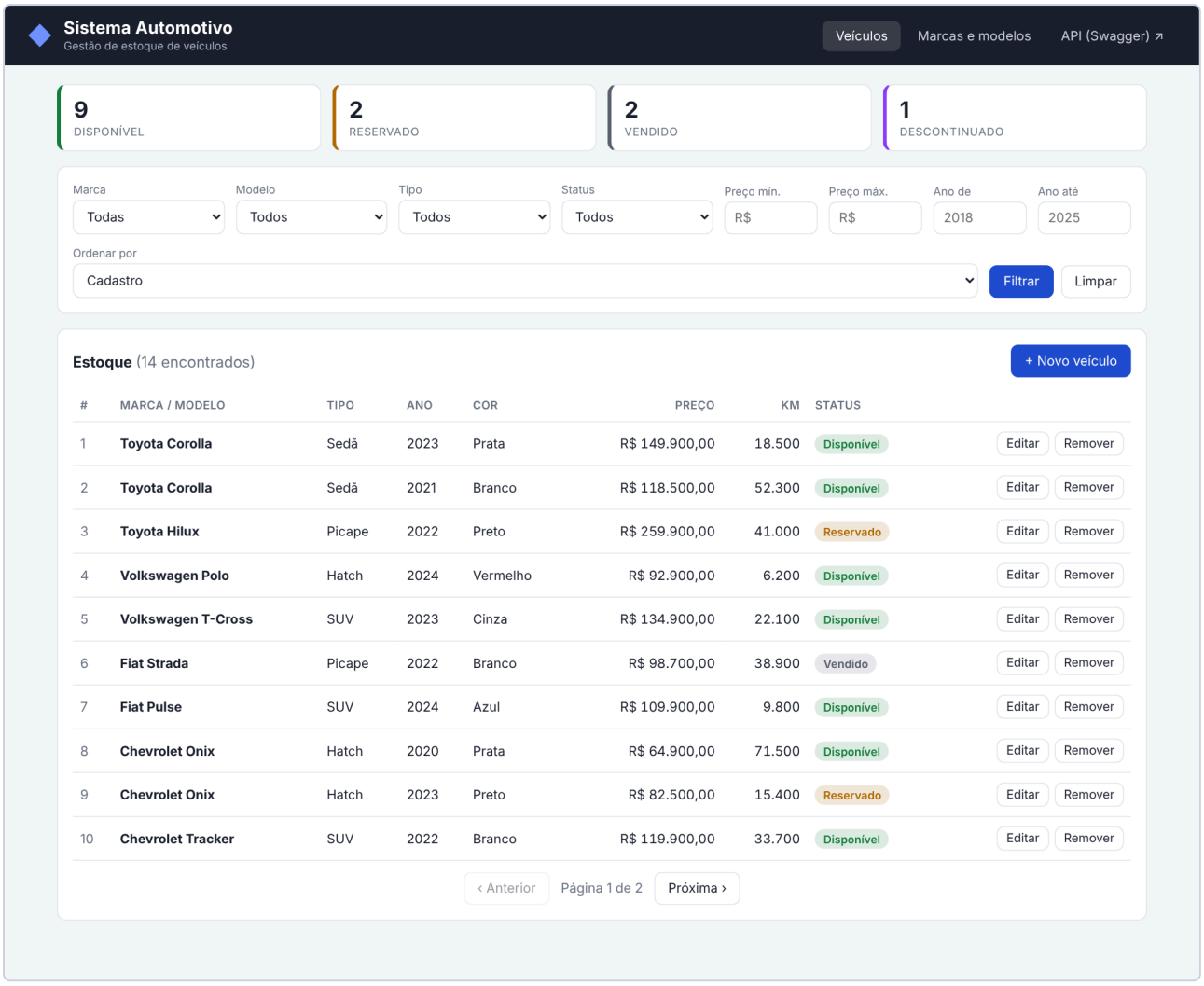
Como executar

Terminal

```
docker compose up --build
# Front-end:  http://localhost:8080
# Swagger:    http://localhost:8080/swagger-ui.html
# MySQL:      localhost:3307 (usuário automotivo / senha automotivo)
```

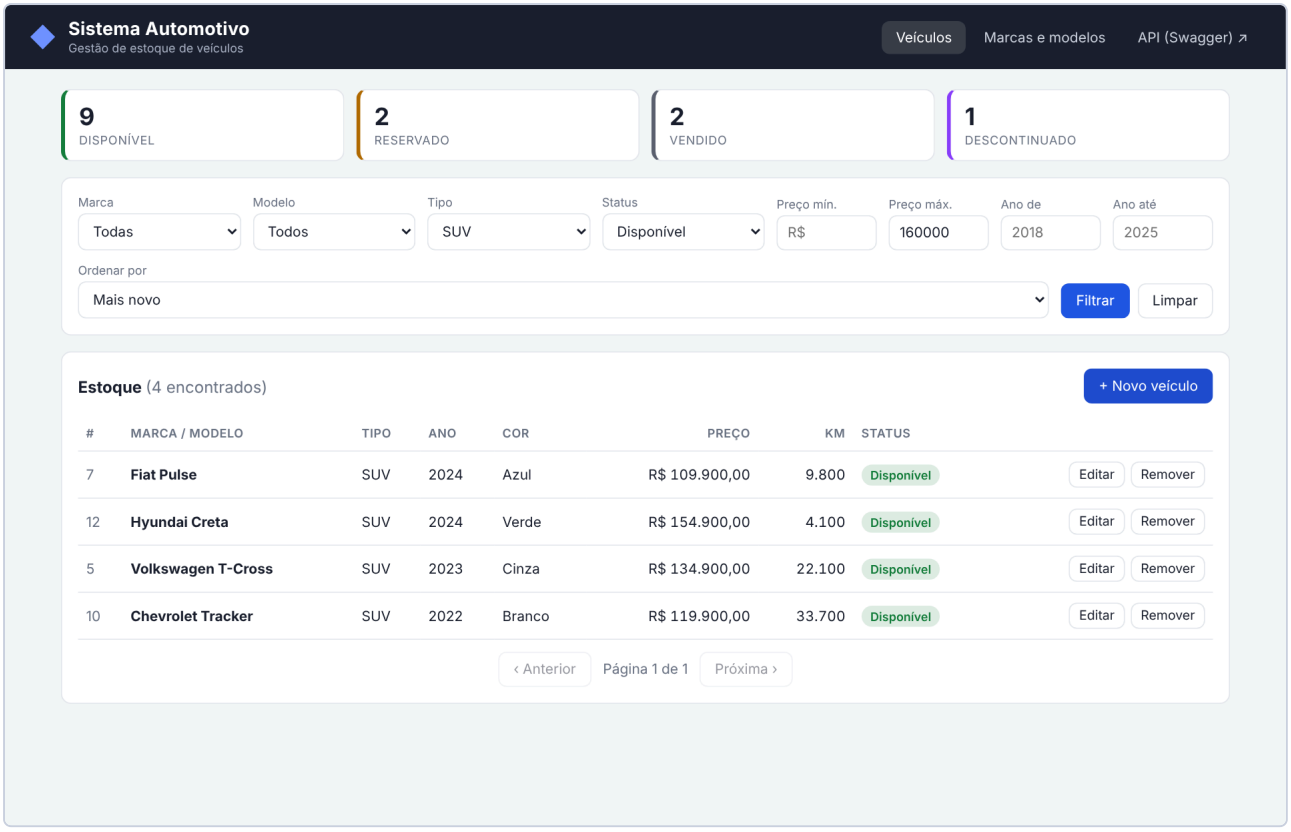
2. Sistema funcionando

2.1 Consulta do estoque (Read)



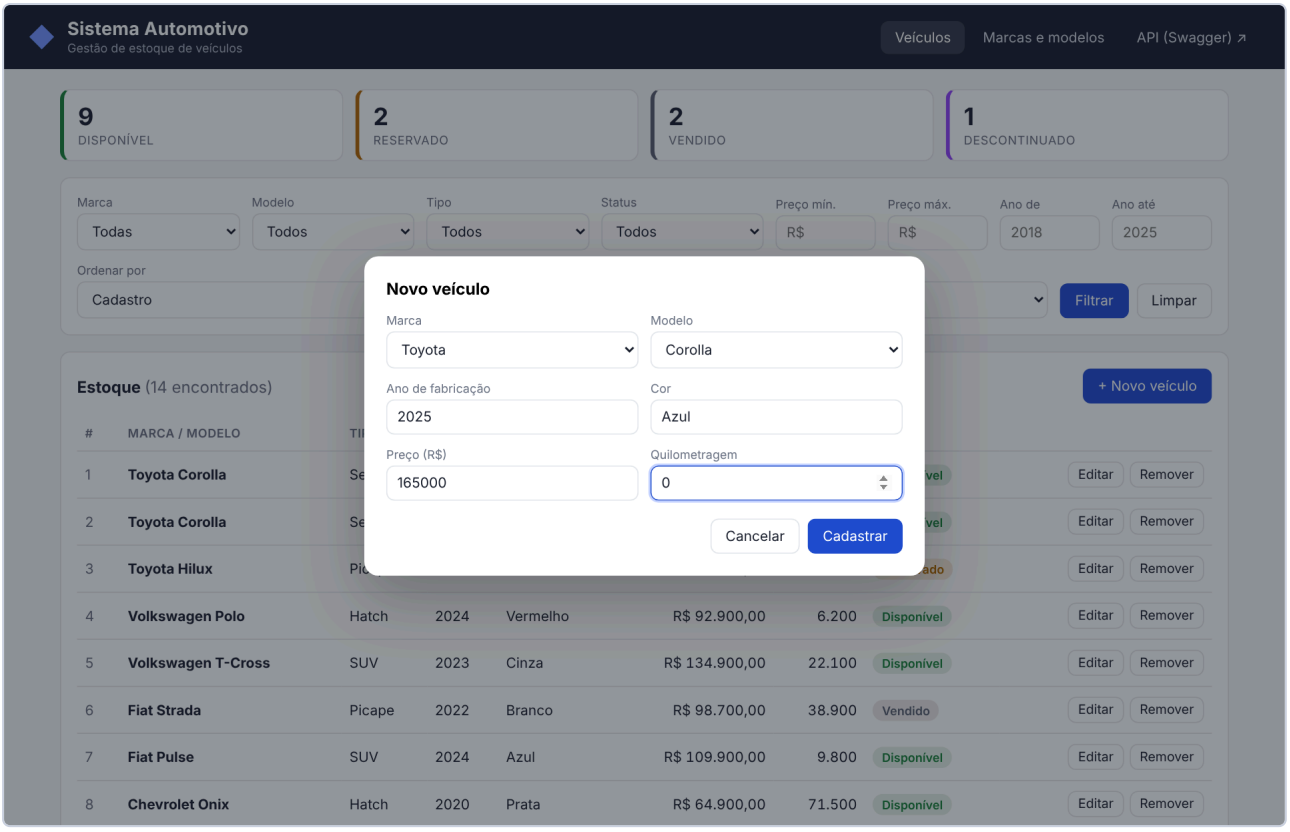
Tela inicial: resumo por status, filtros e listagem paginada do estoque.

2.2 Filtros e consultas (RF03)

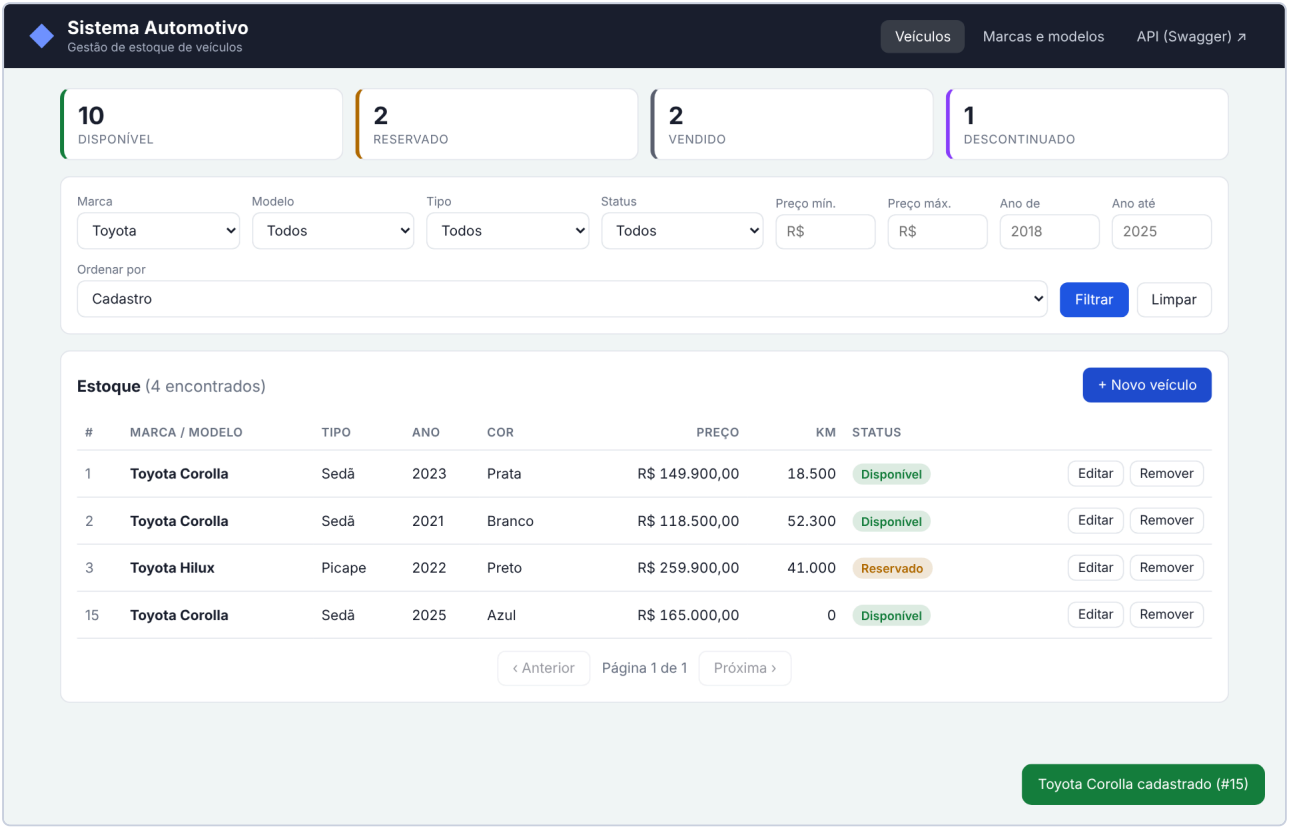


Filtro combinado: SUVs disponíveis até R$ 160.000, ordenados do mais novo para o mais antigo.

2.3 Cadastro (Create)

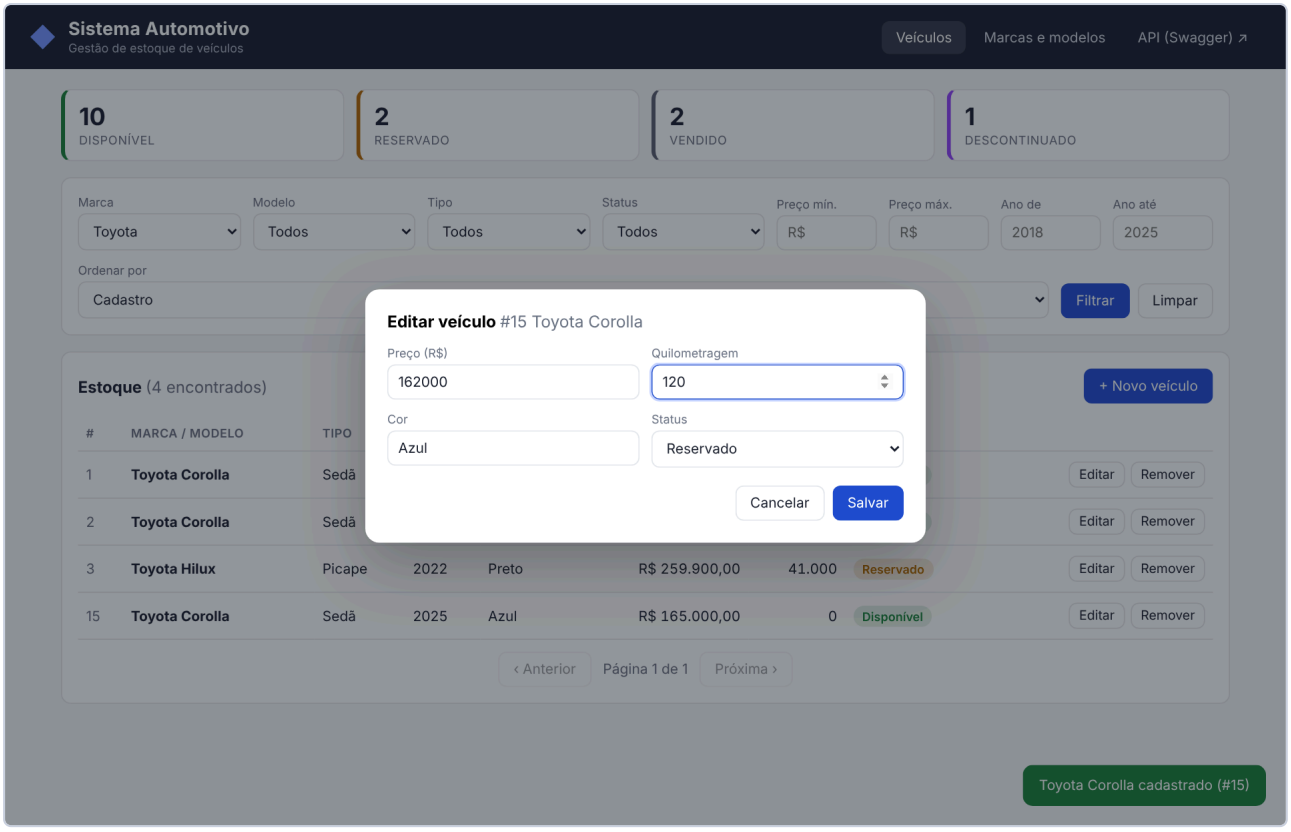


Formulário de cadastro: a marca filtra os modelos disponíveis.

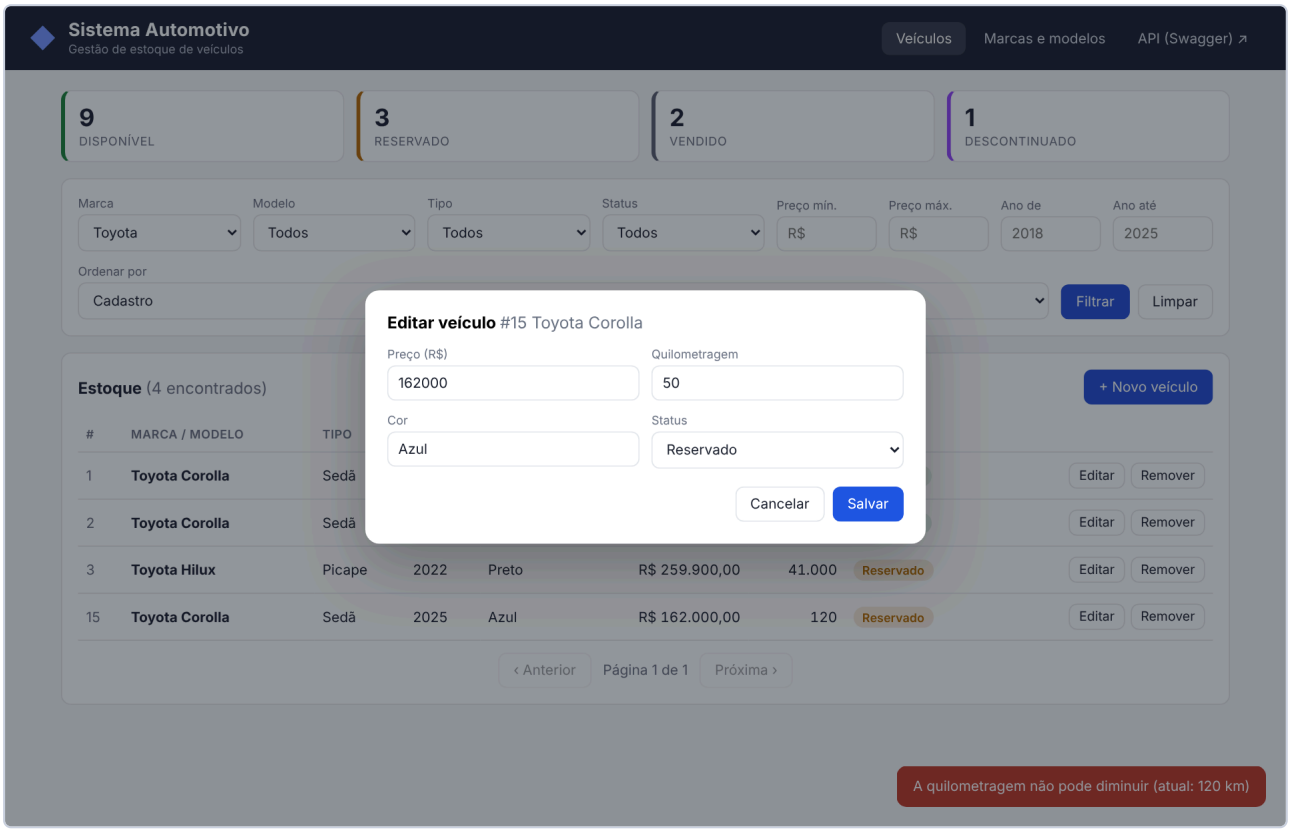


Veículo cadastrado e exibido no estoque como Disponível.

2.4 Atualização (Update)

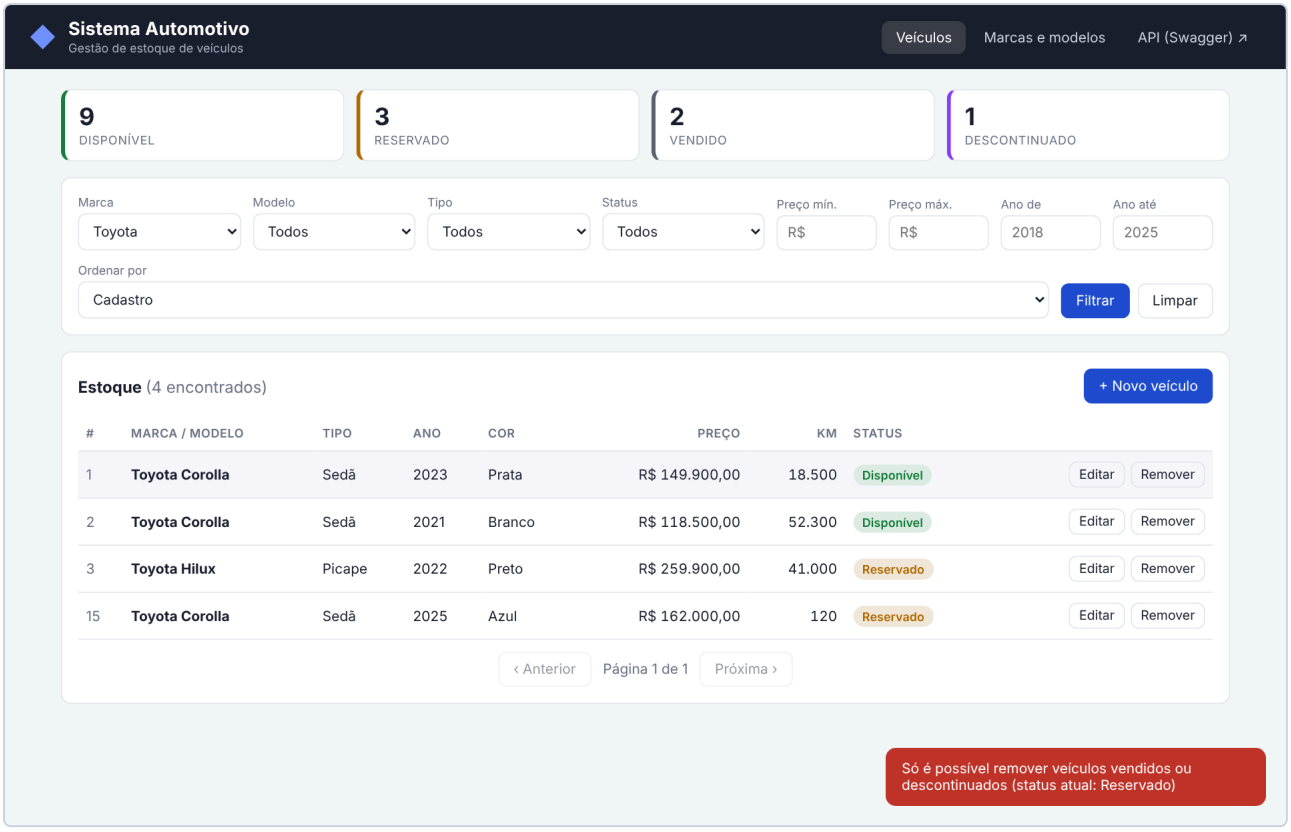


Edição de preço, quilometragem, status e cor (PATCH).

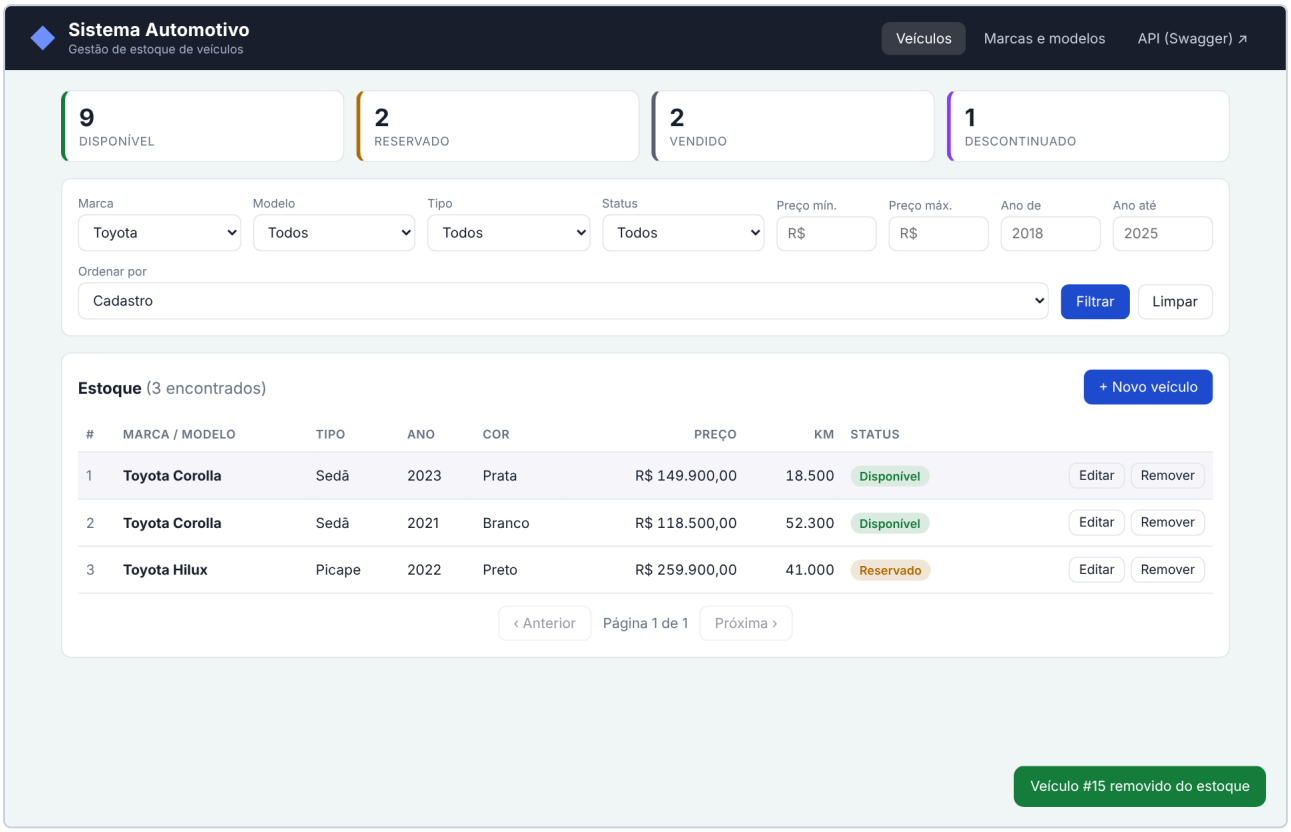


Regra de negócio: a API recusa a redução da quilometragem (HTTP 422).

2.5 Remoção (Delete)

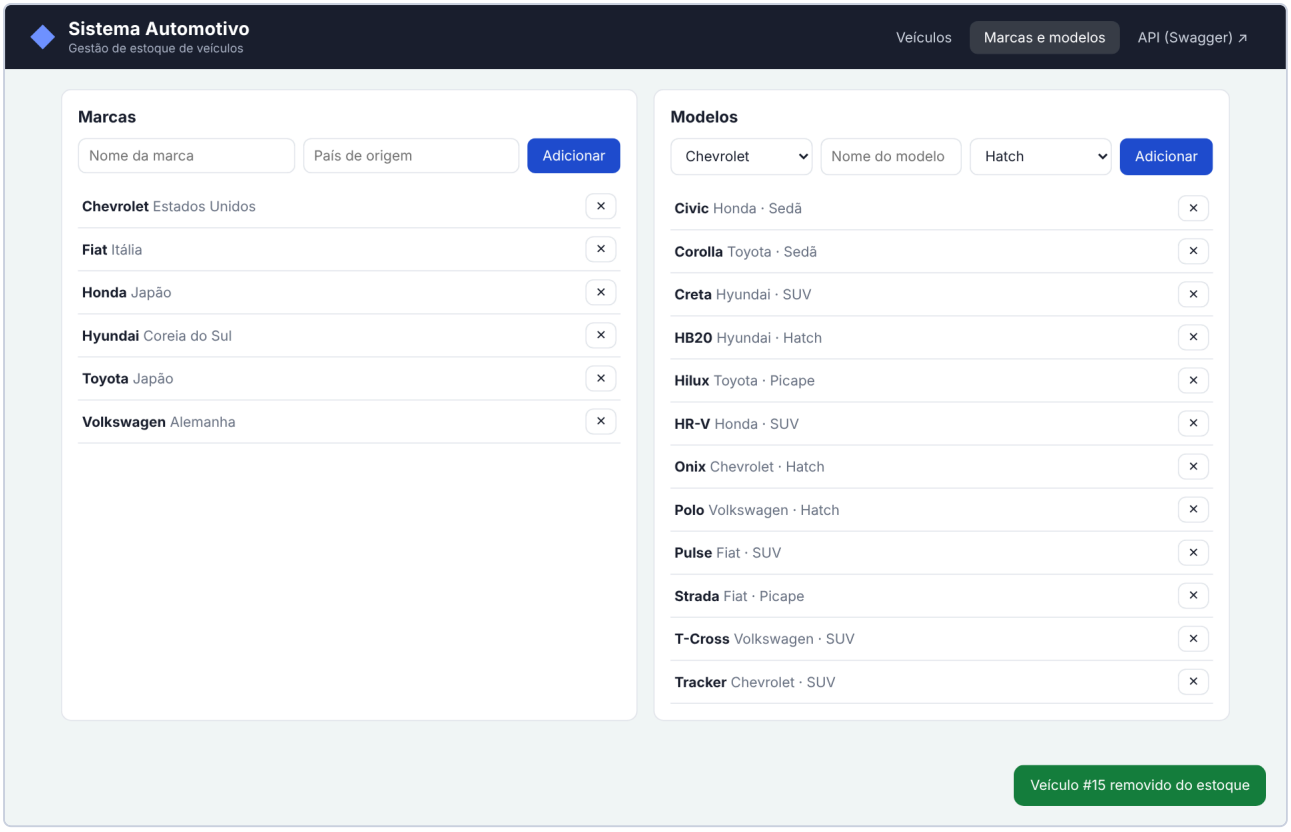


Tentativa de remover um veículo disponível: bloqueada pela regra RN06.



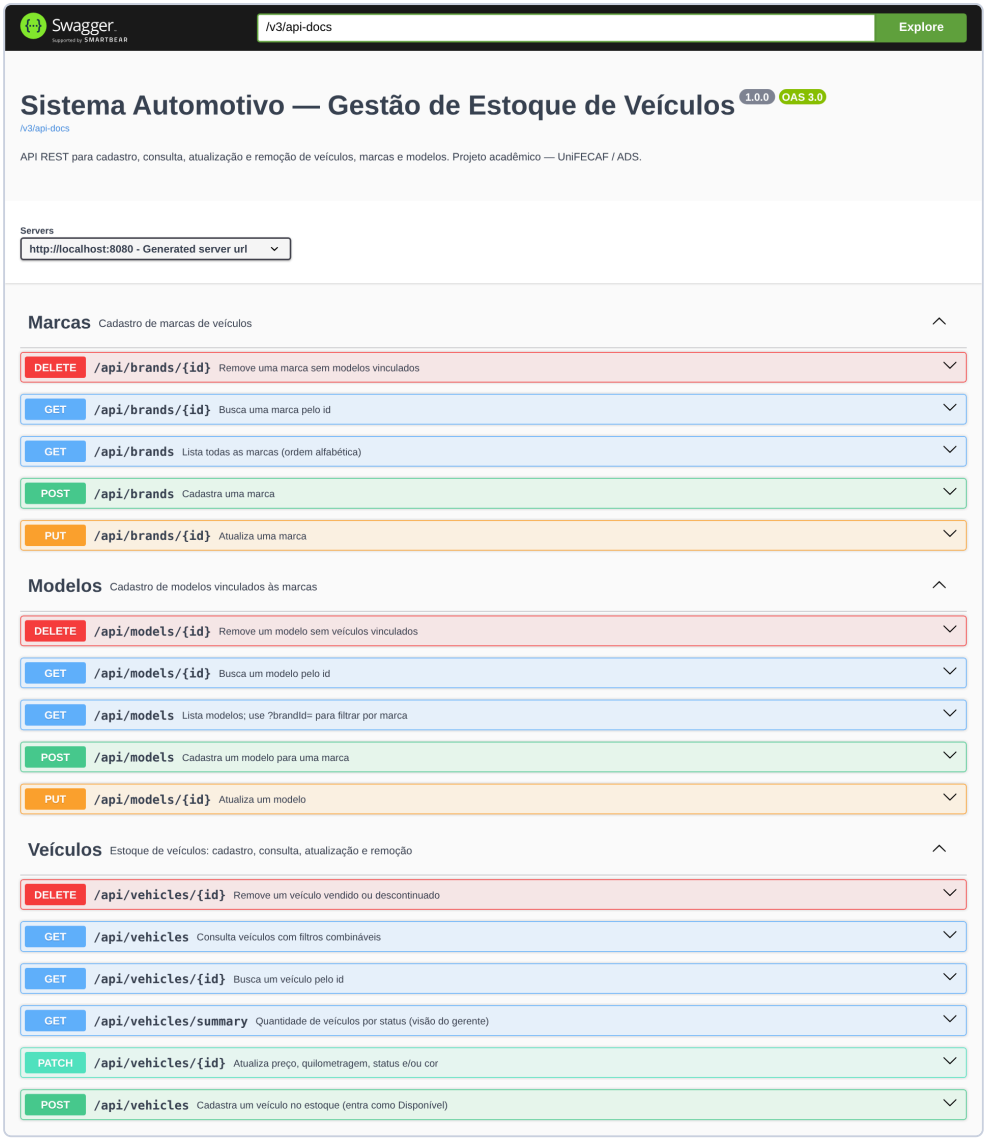
Após marcado como vendido, o veículo é removido do estoque.

2.6 Marcas e modelos (RF02)



Cadastro de marcas e modelos vinculados.

2.7 API REST documentada (Swagger)



Swagger UI com todos os endpoints da API.

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/vehicles?category=SUV&status=AVAILABLE&maxPrice=160000&page=0&size=1&sortBy=id&direction=asc' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/vehicles?category=SUV&status=AVAILABLE&maxPrice=160000&page=0&size=1&sortBy=id&direction=asc
```

Server response

Code	Details
200	Response body


```
{
  "content": [
    {
      "id": 5,
      "brandId": 2,
      "brand": "Volkswagen",
      "modelId": 4,
      "model": "T-Cross",
      "category": "SUV",
      "manufactureYear": 2023,
      "color": "Cinza",
      "price": 134900,
      "mileage": 22100,
      "status": "AVAILABLE",
      "statusLabel": "Disponível",
      "createdAt": "2026-10-15T21:05:48.600537",
      "updatedAt": "2026-10-15T21:05:48.600537"
    }
  ],
  "page": 0,
  "size": 1,
  "totalElements": 4,
  "totalPages": 4
}
```


Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Fri,16 Oct 2026 00:08:22 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Execução de uma consulta filtrada pelo Swagger e a resposta JSON.

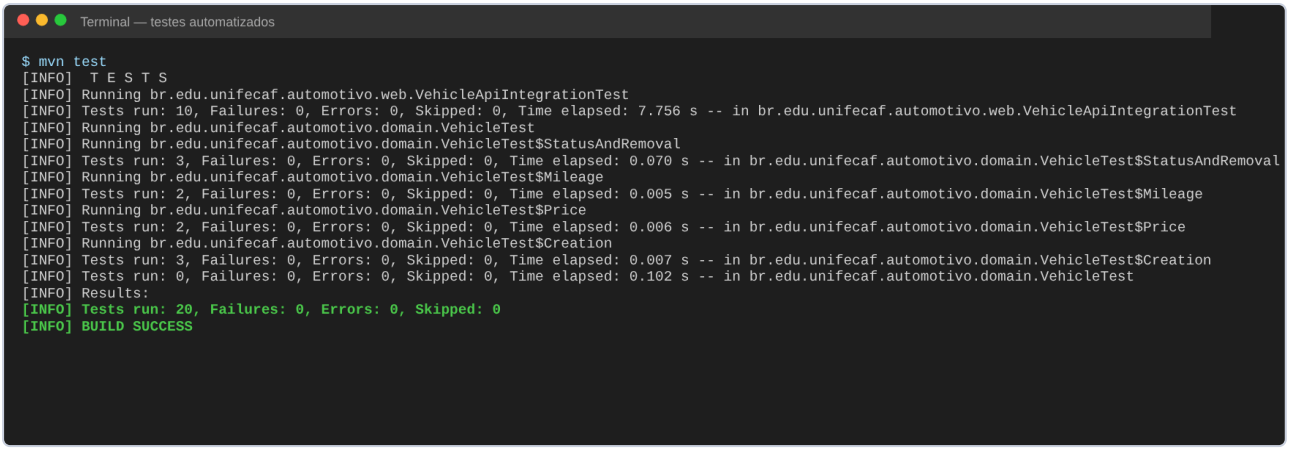
2.8 Banco de dados MySQL

Terminal — MySQL 8 (banco automotivo)

```
mysql> SELECT v.id, b.name AS marca, m.name AS modelo, m.category AS tipo,
       v.manufacture_year AS ano, v.color AS cor, v.price AS preco, v.mileage AS km, v.status
       FROM vehicles v
       JOIN vehicle_models m ON m.id = v.model_id
       JOIN brands b ON b.id = m.brand_id
       ORDER BY v.id;
+----+------------+---------+--------+------+----------+-----------+-------+--------------+
| id | marca      | modelo  | tipo   | ano  | cor      | preco     | km    | status       |
+----+------------+---------+--------+------+----------+-----------+-------+--------------+
|  1 | Toyota     | Corolla | SEDAN  | 2023 | Prata    | 149900.00 | 18500 | AVAILABLE    |
|  2 | Toyota     | Corolla | SEDAN  | 2021 | Branco   | 118500.00 | 52300 | AVAILABLE    |
|  3 | Toyota     | Hilux   | PICKUP | 2022 | Preto    | 259900.00 | 41000 | RESERVED     |
|  4 | Volkswagen | Polo    | HATCH  | 2024 | Vermelho |  92900.00 |  6200 | AVAILABLE    |
|  5 | Volkswagen | T-Cross | SUV    | 2023 | Cinza    | 134900.00 | 22100 | AVAILABLE    |
|  6 | Fiat       | Strada  | PICKUP | 2022 | Branco   |  98700.00 | 38900 | SOLD         |
|  7 | Fiat       | Pulse   | SUV    | 2024 | Azul     | 109900.00 |  9800 | AVAILABLE    |
|  8 | Chevrolet  | Onix    | HATCH  | 2020 | Prata    |  64900.00 | 71500 | AVAILABLE    |
|  9 | Chevrolet  | Onix    | HATCH  | 2023 | Preto    |  82500.00 | 15400 | RESERVED     |
| 10 | Chevrolet  | Tracker | SUV    | 2022 | Branco   | 119900.00 | 33700 | AVAILABLE    |
| 11 | Hyundai    | HB20    | HATCH  | 2019 | Cinza    |  58900.00 | 88200 | DISCONTINUED |
| 12 | Hyundai    | Creta   | SUV    | 2024 | Verde    | 154900.00 |  4100 | AVAILABLE    |
| 13 | Honda      | Civic   | SEDAN  | 2021 | Preto    | 139900.00 | 45600 | SOLD         |
| 14 | Honda      | HR-V    | SUV    | 2023 | Branco   | 162900.00 | 12900 | AVAILABLE    |
+----+------------+---------+--------+------+----------+-----------+-------+--------------+
```

Consulta direta no MySQL mostrando os veículos com seus modelos e marcas (JOIN).

2.9 Testes automatizados



Execução dos testes de unidade e integração com Maven: todos passando.

3. Códigos principais

3.1 Modelo físico (MySQL)

src/main/resources/db/schema.sql

```sql
-- ========================================================================
-- Sistema Automotivo - Modelo físico (MySQL 8)
-- Executado automaticamente na inicialização (spring.sql.init.mode=always).
-- "IF NOT EXISTS" torna o script idempotente: pode rodar a cada start.
-- ========================================================================

CREATE TABLE IF NOT EXISTS brands (
    id          BIGINT       NOT NULL AUTO_INCREMENT,
    name        VARCHAR(80)  NOT NULL,
    country     VARCHAR(60)  NULL,
    created_at  DATETIME(6)  NOT NULL,
    updated_at  DATETIME(6)  NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uk_brands_name UNIQUE (name)
);

CREATE TABLE IF NOT EXISTS vehicle_models (
    id          BIGINT       NOT NULL AUTO_INCREMENT,
    brand_id    BIGINT       NOT NULL,
    name        VARCHAR(80)  NOT NULL,
    category    VARCHAR(20)  NOT NULL,
    created_at  DATETIME(6)  NOT NULL,
    updated_at  DATETIME(6)  NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT uk_models_brand_name UNIQUE (brand_id, name),
    CONSTRAINT fk_models_brand FOREIGN KEY (brand_id) REFERENCES brands (id)
);

CREATE TABLE IF NOT EXISTS vehicles (
    id                BIGINT         NOT NULL AUTO_INCREMENT,
    model_id          BIGINT         NOT NULL,
    manufacture_year  INT            NOT NULL,
    color             VARCHAR(40)    NOT NULL,
    price             DECIMAL(12, 2) NOT NULL,
    mileage           INT            NOT NULL,
    status            VARCHAR(20)    NOT NULL,
    created_at        DATETIME(6)    NOT NULL,
    updated_at        DATETIME(6)    NOT NULL,
    PRIMARY KEY (id),
    CONSTRAINT fk_vehicles_model FOREIGN KEY (model_id) REFERENCES vehicle_models (id),
    CONSTRAINT ck_vehicles_price CHECK (price > 0),
    CONSTRAINT ck_vehicles_mileage CHECK (mileage >= 0),
    -- Índices para os filtros mais usados (requisito não funcional de performance)
    INDEX idx_vehicles_status (status),
    INDEX idx_vehicles_price (price),
    INDEX idx_vehicles_year (manufacture_year)
);
```

3.2 Superclasse das entidades

domain/BaseEntity.java

```java
/**
 * HERANÇA: superclasse abstrata de todas as entidades.
 * Centraliza o identificador e os campos de auditoria (criado/atualizado em),
 * evitando repetir o mesmo código em Brand, VehicleModel e Vehicle.
 *
 * @MappedSuperclass = os campos daqui viram colunas nas tabelas das subclasses
 * (como um "abstract = True" num Model do Django).
 */
@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT do MySQL
    private Long id;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at", nullable = false)
    private LocalDateTime updatedAt;

    /** Callback do JPA executado antes do INSERT. */
    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = this.createdAt;
    }

    /** Callback do JPA executado antes do UPDATE. */
    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    // ENCAPSULAMENTO: só getters. O id e as datas são controlados pelo banco/JPA,
    // nunca alterados diretamente por quem usa o objeto.
    public Long getId() {
        return id;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public LocalDateTime getUpdatedAt() {
        return updatedAt;
    }
}
```

3.3 Entidade Vehicle (regras de negócio)

domain/Vehicle.java

```java
/**
 * Veículo em estoque — a entidade central do sistema.
 *
 * ENCAPSULAMENTO na prática: todos os atributos são private e NÃO existem setters.
 * Toda mudança passa por um método com nome de negócio (changePrice, updateMileage,
 * changeStatus...) que valida a regra antes de alterar o estado. Assim é impossível
 * um veículo ficar com preço negativo ou quilometragem "voltando", venha a chamada
 * da API, do front-end ou de um teste.
 *
```

```java
 * A marca não é guardada aqui: o veículo aponta para o modelo, e o modelo para a marca
 * (Vehicle -> VehicleModel -> Brand), evitando dados duplicados/inconsistentes.
 */
@Entity
@Table(name = "vehicles")
public class Vehicle extends BaseEntity {

    private static final int OLDEST_ACCEPTED_YEAR = 1900;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "model_id", nullable = false)
    private VehicleModel model;

    @Column(name = "manufacture_year", nullable = false)
    private Integer manufactureYear;

    @Column(nullable = false, length = 40)
    private String color;

    @Column(nullable = false, precision = 12, scale = 2) // BigDecimal: nunca use double para
dinheiro
    private BigDecimal price;

    @Column(nullable = false)
    private Integer mileage;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private VehicleStatus status;

    protected Vehicle() {
    }

    /** Todo veículo novo entra no estoque como DISPONÍVEL. */
    public Vehicle(VehicleModel model, int manufactureYear, String color, BigDecimal price, int
mileage) {
        this.status = VehicleStatus.AVAILABLE;
        this.model = Objects.requireNonNull(model, "Modelo é obrigatório");
        this.manufactureYear = validateYear(manufactureYear);
        changeColor(color);
        changePrice(price);
        updateMileage(mileage);
    }

    // ---------------------------------------------------------------------
    // Comportamentos (métodos de negócio)
    // ---------------------------------------------------------------------

    public void changePrice(BigDecimal newPrice) {
        ensureEditable();
        if (newPrice == null || newPrice.signum() <= 0) {
            throw new BusinessRuleException("O preço deve ser maior que zero");
        }
        this.price = newPrice.setScale(2, RoundingMode.HALF_UP);
    }

    public void updateMileage(int newMileage) {
        ensureEditable();
        if (newMileage < 0) {
            throw new BusinessRuleException("A quilometragem não pode ser negativa");
        }
        if (this.mileage != null && newMileage < this.mileage) {
            throw new BusinessRuleException(
                    "A quilometragem não pode diminuir (atual: " + this.mileage + " km)");
        }
        this.mileage = newMileage;
```

```
    }

    public void changeColor(String newColor) {
        ensureEditable();
        if (newColor == null || newColor.isBlank()) {
            throw new BusinessRuleException("A cor é obrigatória");
        }
        this.color = newColor.trim();
    }

    public void changeStatus(VehicleStatus newStatus) {
        Objects.requireNonNull(newStatus, "Status é obrigatório");
        if (!status.canTransitionTo(newStatus)) {
            throw new BusinessRuleException(
                    "Não é permitido mudar o status de " + status.getLabel() + " para " +
newStatus.getLabel());
        }
        this.status = newStatus;
    }

    public void markAsSold() {
        changeStatus(VehicleStatus.SOLD);
    }

    public boolean canBeRemoved() {
        return status.allowsRemoval();
    }

    /** Lança exceção se o veículo ainda está em estoque ativo (regra do requisito 5). */
    public void ensureCanBeRemoved() {
        if (!canBeRemoved()) {
            throw new BusinessRuleException(
                    "Só é possível remover veículos vendidos ou descontinuados (status atual: "
                            + status.getLabel() + ")");
        }
    }

    // ------------------------------------------------------------------
    // Métodos auxiliares privados (detalhes internos escondidos)
    // ------------------------------------------------------------------

    private void ensureEditable() {
        if (status != null && !status.allowsEditing()) {
            throw new BusinessRuleException("Veículo vendido não pode ser editado");
        }
    }

    private static int validateYear(int year) {
        int maxYear = Year.now().getValue() + 1; // ano-modelo pode ser o próximo
        if (year < OLDEST_ACCEPTED_YEAR || year > maxYear) {
            throw new BusinessRuleException(
                    "Ano de fabricação deve estar entre " + OLDEST_ACCEPTED_YEAR + " e " + maxYear);
        }
        return year;
    }

    // ... getters omitidos ...
}
```

3.4 Filtros dinâmicos

repository/VehicleSpecifications.java

```java
/**
 * Monta o WHERE dinamicamente: só entra na consulta o filtro que foi informado.
 * Equivale a ir encadeando .filter() num QuerySet do Django.
 */
public final class VehicleSpecifications {

    private VehicleSpecifications() {
        // classe utilitária: não deve ser instanciada
    }

    public static Specification<Vehicle> matching(VehicleFilter filter) {
        return (root, query, cb) -> {
            List<Predicate> predicates = new ArrayList<>();
            Join<Vehicle, VehicleModel> model = root.join("model");
            Join<VehicleModel, Brand> brand = model.join("brand");

            if (filter.brandId() != null) {
                predicates.add(cb.equal(brand.get("id"), filter.brandId()));
            }
            if (filter.modelId() != null) {
                predicates.add(cb.equal(model.get("id"), filter.modelId()));
            }
            if (hasText(filter.brand())) {
                predicates.add(cb.like(cb.lower(brand.get("name")), contains(filter.brand())));
            }
            if (hasText(filter.model())) {
                predicates.add(cb.like(cb.lower(model.get("name")), contains(filter.model())));
            }
            if (filter.category() != null) {
                predicates.add(cb.equal(model.get("category"), filter.category()));
            }
            if (filter.status() != null) {
                predicates.add(cb.equal(root.get("status"), filter.status()));
            }
            if (filter.minPrice() != null) {
                predicates.add(cb.greaterThanOrEqualTo(root.get("price"), filter.minPrice()));
            }
            if (filter.maxPrice() != null) {
                predicates.add(cb.lessThanOrEqualTo(root.get("price"), filter.maxPrice()));
            }
            if (filter.minYear() != null) {
                predicates.add(cb.greaterThanOrEqualTo(root.get("manufactureYear"),
filter.minYear()));
            }
            if (filter.maxYear() != null) {
                predicates.add(cb.lessThanOrEqualTo(root.get("manufactureYear"), filter.maxYear()));
            }
            if (hasText(filter.color())) {
                predicates.add(cb.like(cb.lower(root.get("color")), contains(filter.color())));
            }
            return cb.and(predicates.toArray(Predicate[]::new));
        };
    }

    private static boolean hasText(String value) {
        return value != null && !value.isBlank();
    }

    private static String contains(String value) {
        return "%" + value.trim().toLowerCase(Locale.ROOT) + "%";
    }
}
```

repository/VehicleRepository.java

```java
/**
 * JpaSpecificationExecutor permite montar consultas dinâmicas (filtros opcionais)
 * com a Criteria API — ver VehicleSpecifications.
 */
public interface VehicleRepository extends JpaRepository<Vehicle, Long>,
JpaSpecificationExecutor<Vehicle> {

    @Override
    @EntityGraph(attributePaths = {"model", "model.brand"})
    Page<Vehicle> findAll(Specification<Vehicle> spec, Pageable pageable);

    @Override
    @EntityGraph(attributePaths = {"model", "model.brand"})
    Optional<Vehicle> findById(Long id);

    boolean existsByModelId(Long modelId);

    long countByStatus(VehicleStatus status);
}
```

3.5 Serviço

service/VehicleService.java

```java
/** Casos de uso do estoque de veículos (RF01, RF03, RF04, RF05). */
@Service
@Transactional
public class VehicleService {

    private final VehicleRepository vehicles;
    private final VehicleModelService modelService;

    public VehicleService(VehicleRepository vehicles, VehicleModelService modelService) {
        this.vehicles = vehicles;
        this.modelService = modelService;
    }

    /** RF03 — consulta com filtros combináveis + paginação/ordenação. */
    @Transactional(readOnly = true)
    public PageResponse<VehicleResponse> search(VehicleFilter filter, Pageable pageable) {
        filter.validate();
        var page = vehicles.findAll(VehicleSpecifications.matching(filter), pageable)
                .map(VehicleResponse::from);
        return PageResponse.from(page);
    }

    @Transactional(readOnly = true)
    public VehicleResponse findById(Long id) {
        return VehicleResponse.from(getOrThrow(id));
    }

    /** Painel do gerente: quantos veículos em cada status. */
    @Transactional(readOnly = true)
    public Map<VehicleStatus, Long> countByStatus() {
        Map<VehicleStatus, Long> summary = new EnumMap<>(VehicleStatus.class);
        for (VehicleStatus status : VehicleStatus.values()) {
            summary.put(status, vehicles.countByStatus(status));
        }
        return summary;
    }
```

```
    /** RF01 — cadastro. */
    public VehicleResponse create(VehicleCreateRequest request) {
        VehicleModel model = modelService.getOrThrow(request.modelId());
        Vehicle vehicle = new Vehicle(
                model,
                request.manufactureYear(),
                request.color(),
                request.price(),
                request.mileage());
        return VehicleResponse.from(vehicles.save(vehicle));
    }

    /** RF04 — atualização parcial. As regras ficam nos métodos da entidade. */
    public VehicleResponse update(Long id, VehicleUpdateRequest request) {
        Vehicle vehicle = getOrThrow(id);
        if (request.color() != null) {
            vehicle.changeColor(request.color());
        }
        if (request.price() != null) {
            vehicle.changePrice(request.price());
        }
        if (request.mileage() != null) {
            vehicle.updateMileage(request.mileage());
        }
        if (request.status() != null) { // por último: permite ajustar preço e marcar como vendido no
mesmo PATCH
            vehicle.changeStatus(request.status());
        }
        // saveAndFlush força o UPDATE agora, para devolver o updatedAt já atualizado
        return VehicleResponse.from(vehicles.saveAndFlush(vehicle));
    }

    /** RF05 — remoção (somente vendidos ou descontinuados). */
    public void delete(Long id) {
        Vehicle vehicle = getOrThrow(id);
        vehicle.ensureCanBeRemoved();
        vehicles.delete(vehicle);
    }

    private Vehicle getOrThrow(Long id) {
        return vehicles.findById(id).orElseThrow(() -> new ResourceNotFoundException("Veículo", id));
    }
}
```

3.6 Controller REST

web/VehicleController.java

```
/**
 * Controller REST do estoque de veículos.
 *
 *   GET    /api/vehicles          consulta com filtros (RF03)
 *   GET    /api/vehicles/{id}     detalhe
 *   GET    /api/vehicles/summary  total por status
 *   POST   /api/vehicles          cadastro (RF01)
 *   PATCH  /api/vehicles/{id}     atualização de preço/km/status/cor (RF04)
 *   DELETE /api/vehicles/{id}     remoção de vendidos/descontinuados (RF05)
 */
@RestController
@RequestMapping("/api/vehicles")
@Tag(name = "Veículos", description = "Estoque de veículos: cadastro, consulta, atualização e
remoção")
```

```java
public class VehicleController {

    private static final Set<String> SORTABLE_FIELDS =
            Set.of("id", "price", "manufactureYear", "mileage", "color", "status", "createdAt");
    private static final int MAX_PAGE_SIZE = 100;

    private final VehicleService service;

    public VehicleController(VehicleService service) {
        this.service = service;
    }

    @GetMapping
    @Operation(summary = "Consulta veículos com filtros combináveis",
            description = "Filtros: brandId, modelId, brand (nome), model (nome), category, status, "
                    + "minPrice, maxPrice, minYear, maxYear, color. Paginação: page, size. "
                    + "Ordenação: sortBy (price, manufactureYear, mileage...) e direction
(asc/desc).")
    public PageResponse<VehicleResponse> search(
            @ParameterObject VehicleFilter filter,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size,
            @RequestParam(defaultValue = "id") String sortBy,
            @RequestParam(defaultValue = "asc") String direction) {
        return service.search(filter, buildPageable(page, size, sortBy, direction));
    }

    @GetMapping("/{id}")
    @Operation(summary = "Busca um veículo pelo id")
    public VehicleResponse get(@PathVariable Long id) {
        return service.findById(id);
    }

    @GetMapping("/summary")
    @Operation(summary = "Quantidade de veículos por status (visão do gerente)")
    public Map<VehicleStatus, Long> summary() {
        return service.countByStatus();
    }

    @PostMapping
    @Operation(summary = "Cadastra um veículo no estoque (entra como Disponível)")
    public ResponseEntity<VehicleResponse> create(@Valid @RequestBody VehicleCreateRequest request) {
        VehicleResponse created = service.create(request);
        return ResponseEntity.created(URI.create("/api/vehicles/" + created.id())).body(created);
    }

    @PatchMapping("/{id}")
    @Operation(summary = "Atualiza preço, quilometragem, status e/ou cor")
    public VehicleResponse update(@PathVariable Long id, @Valid @RequestBody VehicleUpdateRequest
request) {
        return service.update(id, request);
    }

    @DeleteMapping("/{id}")
    @Operation(summary = "Remove um veículo vendido ou descontinuado")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }

    /** Lista branca de campos ordenáveis: evita erro 500 com ?sortBy=campoInexistente. */
    private static Pageable buildPageable(int page, int size, String sortBy, String direction) {
        if (!SORTABLE_FIELDS.contains(sortBy)) {
            throw new IllegalArgumentException("sortBy inválido. Use um destes: " + SORTABLE_FIELDS);
        }
        if (page < 0 || size < 1 || size > MAX_PAGE_SIZE) {
```

```
            throw new IllegalArgumentException("page deve ser >= 0 e size entre 1 e " +
MAX_PAGE_SIZE);
        }
        Sort.Direction dir = "desc".equalsIgnoreCase(direction) ? Sort.Direction.DESC :
Sort.Direction.ASC;
        return PageRequest.of(page, size, Sort.by(dir, sortBy));
    }
}
```

3.7 Tratamento global de erros

web/GlobalExceptionHandler.java

```
/**
 * Converte exceções em respostas HTTP padronizadas (RFC 9457 "Problem Details").
 * Assim os controllers ficam limpos e todo erro tem o mesmo formato JSON:
 * { "status": 422, "title": "...", "detail": "..." }
 */
@RestControllerAdvice
@Order(Ordered.HIGHEST_PRECEDENCE) // tem prioridade sobre os tratadores padrão do Spring
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        return problem(HttpStatus.NOT_FOUND, "Recurso não encontrado", ex.getMessage());
    }

    @ExceptionHandler(BusinessRuleException.class)
    public ProblemDetail handleBusinessRule(BusinessRuleException ex) {
        return problem(HttpStatus.UNPROCESSABLE_ENTITY, "Regra de negócio violada", ex.getMessage());
    }

    /** Erros de @Valid no corpo (MethodArgumentNotValidException é subclasse) e de binding na query.
*/
    @ExceptionHandler(BindException.class)
    public ProblemDetail handleValidation(BindException ex) {
        Map<String, String> errors = new LinkedHashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.putIfAbsent(error.getField(), error.getDefaultMessage());
        }
        ProblemDetail problem = problem(HttpStatus.BAD_REQUEST, "Dados inválidos",
                "Um ou mais campos não passaram na validação");
        problem.setProperty("errors", errors);
        return problem;
    }

    @ExceptionHandler({
            IllegalArgumentException.class,
            HttpMessageNotReadableException.class,
            MethodArgumentTypeMismatchException.class
    })
    public ProblemDetail handleBadRequest(Exception ex) {
        String detail = (ex instanceof HttpMessageNotReadableException)
                ? "JSON inválido ou valor fora do esperado (verifique enums como status e category)"
                : ex.getMessage();
        return problem(HttpStatus.BAD_REQUEST, "Requisição inválida", detail);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ProblemDetail handleIntegrity(DataIntegrityViolationException ex) {
        return problem(HttpStatus.CONFLICT, "Conflito de dados",
                "A operação viola uma restrição do banco (registro duplicado ou em uso)");
    }
```

```
    private static ProblemDetail problem(HttpStatus status, String title, String detail) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(status, detail);
        problem.setTitle(title);
        return problem;
    }
}
```

3.8 Testes de unidade (trecho)

test/.../domain/VehicleTest.java

```
/**
 * Testes de UNIDADE do domínio: Java puro, sem Spring e sem banco — rodam em milissegundos.
 * Cada teste documenta uma regra de negócio da entidade Vehicle.
 */
@DisplayName("Vehicle (regras de negócio)")
class VehicleTest {

    private VehicleModel corolla;
    private Vehicle vehicle;

    @BeforeEach
    void setUp() {
        corolla = new VehicleModel("Corolla", VehicleCategory.SEDAN, new Brand("Toyota", "Japão"));
        vehicle = new Vehicle(corolla, 2022, "Prata", new BigDecimal("120000"), 30000);
    }

    @Nested
    @DisplayName("Cadastro")
    class Creation {

        @Test
        @DisplayName("todo veículo novo entra como DISPONÍVEL")
        void newVehicleStartsAvailable() {
            assertThat(vehicle.getStatus()).isEqualTo(VehicleStatus.AVAILABLE);
            assertThat(vehicle.getBrand().getName()).isEqualTo("Toyota");
            assertThat(vehicle.getPrice()).isEqualByComparingTo("120000.00");
        }

        @Test
        @DisplayName("rejeita ano de fabricação fora do intervalo aceito")
        void rejectsInvalidYear() {
            int tooFarAhead = Year.now().getValue() + 2;
            assertThatThrownBy(() -> new Vehicle(corolla, tooFarAhead, "Prata", BigDecimal.TEN, 0))
                    .isInstanceOf(BusinessRuleException.class)
                    .hasMessageContaining("Ano de fabricação");
            assertThatThrownBy(() -> new Vehicle(corolla, 1899, "Prata", BigDecimal.TEN, 0))
                    .isInstanceOf(BusinessRuleException.class);
        }

        @Test
        @DisplayName("rejeita cor em branco")
        void rejectsBlankColor() {
            assertThatThrownBy(() -> new Vehicle(corolla, 2022, "  ", BigDecimal.TEN, 0))
                    .isInstanceOf(BusinessRuleException.class);
        }
    }

    // ... demais testes no repositório ...
}
```